

Programming in Modern C++: Assignment Week 11

Total Marks : 25

Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur, Kharagpur – 721302
partha.p.das@gmail.com

September 28, 2025

Question 1

Consider the program (in C++11) given below.

[MSQ, Marks 2]

```
#include <iostream>
#include <string>
#include <utility>

class X {
public:
    X(const std::string& s) { std::cout << "copy : " << s << std::endl; }
    X(std::string&& s) { std::cout << "move : " << s << std::endl; }
};

class Y {
public:
    template<typename T, typename U>
    Y(T&& s1, U&& s2) : ----- { }    //LINE-1
private:
    X x1_, x2_;
};

int main(){
    std::string str = "Hello";
    Y y1(str, std::string("World"));
    return 0;
}
```

Choose the appropriate option to fill in the blank at LINE-1 so that the output becomes

copy : Hello

move : World

- a) LINE-1: x1_(s1), x2_(s2)
- b) LINE-1: x1_(std::forward<T>(s1)), x2_(std::forward<U>(s2))
- c) LINE-1: x1_(s1), x2_(std::forward<U>(s2))
- d) LINE-1: x1_(std::forward<T>(s1)), x2_(s2)

Answer: b), c)

Explanation:

From the output we can understand that for object `x1` the copy version of the constructor and for object `x2` the move version of the constructor need to be called. Since `s1` and `s2` are universal references, `s1` can be forwarded as `x1_(s1)` or `x1_(std::forward<T>(s1))`. But `s2` must be forwarded as an r-value reference: `x2_(std::forward<U>(s2))`.

Question 2

Consider the code segment (in C++11) given below.

[MSQ, Marks 2]

```
double&& x = 3.14;           //LINE-1
auto&& y = x;                 //LINE-2
```

```
template<typename U>
void foo(U&& v);              //LINE-3
```

```
template<typename V>
void bar(std::string<V>&& v);  //LINE-4
```

Identify the line/s where `&&` indicates a universal reference.

- a) LINE-1
- b) LINE-2
- c) LINE-3
- d) LINE-4

Answer: b), c)

Explanation:

- At LINE-1, `double&&` is declared explicitly, so it is an rvalue reference, not a universal reference.
 - At LINE-2, `auto` requires type deduction, so `auto&&` is a universal reference.
 - At LINE-3, since `U` is deduced, `U&&` is a universal reference.
 - At LINE-4, even though `V` is deduced, the parameter type is not of the form `V&&`; it is `std::string<V>&&`, which is always an rvalue reference.
- Therefore, LINE-2 and LINE-3 indicate universal references.

Question 3

Consider the code segment (in C++11) given below.

[MSQ, Marks 2]

```
#include <iostream>
#include <cstring>

class MyString {
public:
    MyString() = delete;
    MyString(const char* s) {
        if (s) {
            data = new charstd::[strlen(s) + 1];
            std::strcpy(data, s);
        } else {
            data = nullptr;
        }
    }
    MyString(const MyString& other) {
        if (other.data) {
            data = new char[strlen(other.data) + 1];
            std::strcpy(data, other.data);
        } else {
            data = nullptr;
        }
    }
    MyString& operator=(const MyString& other) = delete;
    ~MyString() { delete[] data; }
    void show(){ if (data) std::cout << data; }
private:
    char* data;
};

int main(){
    MyString s1;           // LINE-1
    MyString s2("Hello");  // LINE-2
    MyString s3 = s2;       // LINE-3
    s2 = s3;               // LINE-4
    return 0;
}
```

Identify the line(s) that generate(s) error during compilation.

- a) LINE-1
- b) LINE-2
- c) LINE-3
- d) LINE-4

Answer: a), d)

Explanation:

Since the default constructor `MyString()` is deleted, LINE-1 generates a compiler error.

Since the copy assignment operator is also deleted, LINE-4 generates a compiler error.

LINE-2 and LINE-3 work correctly because constructor and copy constructor are available.

Question 4

Consider the following code segment (in C++11).

[MSQ, Marks 2]

```
#include <iostream>

enum class PLANET {EARTH, MARS, JUPITER};
enum class GOD {MARS, JUPITER, NEPTUNE};
enum MYTHOLOGY {EARTH, MARS, JUPITER};

bool isEarth(PLANET p){
    if(p == PLANET::EARTH)           //LINE-1
        return true;
    return false;
}

bool isMars(PLANET p){
    if(p == GOD::MARS)               //LINE-2
        return true;
    return false;
}

bool isJupiter(PLANET p){
    if(p == JUPITER)                 //LINE-3
        return true;
    return false;
}

int main() {
    if(isEarth(PLANET::EARTH) && isMars(PLANET::MARS) && isJupiter(PLANET::JUPITER))
        std::cout << "true";
    return 0;
}
```

Identify the statement/s which are true for the given program.

- a) It generates compiler error at LINE-1
- b) It generates compiler error at LINE-2
- c) It generates compiler error at LINE-3
- d) It generates the output `true`

Answer: b), c)

Explanation:

The statement `if(p == PLANET::EARTH)` compares two `PLANET` elements, which compiles successfully.

The statement `if(p == GOD::MARS)` compares `PLANET` with `GOD`, which are unrelated scoped enums, hence a compiler error.

The statement `if(p == JUPITER)` compares `PLANET` with `int` (from unscoped enum `MYTHOLOGY`), hence a compiler error.

Since the code generates compiler errors, it will not produce any output.

Question 5

Consider the following code segment (in C++11).

[MCQ, Marks 2]

```
#include <iostream>
#include <string>

class book {
public:
    explicit book() : book("Unknown") { std::cout << "#A "; } // LINE-1
    explicit book(const std::string& t) : book(t, dPrice)
    { std::cout << "#B "; } // LINE-2
    explicit book(double p) : book("Unknown", p)
    { std::cout << "#C "; } // LINE-3
    book(const std::string& t, double p) : title_{t}, price_{p}
    { std::cout << "#D "; } // LINE-4
private:
    std::string title_ {" "};
    double price_ {0.0};
    static constexpr double dPrice = 499.99;
};

int main() {
    book b1;
    return 0;
}
```

What will be the output?

- a) #A
- b) #B #A
- c) #D #B #A
- d) #A #B #D

Answer: c)

Explanation:

The instantiation `book b1;` triggers the following constructor calls in order:

1. constructor at LINE-1,
2. constructor at LINE-2,
3. constructor at LINE-4.

Since constructor bodies are executed in reverse order of delegation, the output is `#D #B #A`.

Question 6

Consider the following program (in C++14).

[MCQ, Marks 2]

```
#include <iostream>
#include <string>

template<typename T> T factor = T(10);

int main() {
    factor<int> = 2;
    auto scale_int = [](auto v) { return v * factor<decltype(v)>; }(5);
    auto scale_double = [](auto v) { return v * factor<decltype(v)>; }(5.0);
    std::cout << scale_int << ", " << scale_double;
    return 0;
}
```

What will be the output?

- a) 50, 50
- b) 10, 50
- c) 10, 5
- d) 10, 50.0

Answer: b)

Explanation:

In the expression: `[](auto v) { return v * factor<decltype(v)>; }(5)`, the inferred type of `v` is `int`. Since `factor<int> = 2`, the result is 10.

In the expression: `[](auto v) { return v * factor<decltype(v)>; }(5.0)`, the inferred type of `v` is `double`. Since `factor<double> = 10.0`, the result is 50.

Hence, the output is 10, 50.

Question 7

Consider the following λ expression (in C++14).

[MCQ, Marks 2]

```
auto mul_add = [a, &b](int x, int y) {  
    b += x * y;  
    return a + b;  
};
```

Identify the most appropriate option that defines the equivalent closure object for the above lambda function.

- a)

```
struct mul_add_closure {  
    int a;  
    int b;  
    mul_add_closure(int a_, int& b_) : a(a_), b(b_) { }  
    int operator()(int x, int y) const {  
        b += x * y;  
        return a + b;  
    }  
};  
auto mul_add = mul_add_closure(a, b);
```
- b)

```
struct mul_add_closure {  
    int a;  
    int& b;  
    mul_add_closure(int a_, int& b_) : a(a_), b(b_) { }  
    int operator()(int x, int y) const {  
        b += x * y;  
        return a + b;  
    }  
};  
auto mul_add = mul_add_closure(a, b);
```
- c)

```
struct mul_add_closure {  
    int& a;  
    int& b;  
    mul_add_closure(int& a_, int& b_) : a(a_), b(b_) { }  
    int operator()(int x, int y) const {  
        b += x * y;  
        return a + b;  
    }  
};  
auto mul_add = mul_add_closure(a, b);
```
- d)

```
struct mul_add_closure {  
    int a;  
    int& b;  
    mul_add_closure(int a_, int& b_) : a(a_), b(b_) { }  
    void operator()(int x, int y, int& res) const {  
        b += x * y;  
        res = a + b;  
    }  
};  
auto mul_add = mul_add_closure(a, b);
```

Answer: b)

Explanation:

For a λ -expression, the compiler generates a functor class with:

- data members:
 - value member for each value capture (**a**)
 - reference member for each reference capture (**b**)
- a constructor that initializes captured variables appropriately
- an `operator()` method that executes the lambda body
- implicit copy/move constructors, assignment operators, and destructor

Question 8

Consider the following code segment (in C++11).

[MSQ, Marks 2]

```
#include <iostream>
#include <string>

class floatData {
public:
    floatData(int data) : d1_(data) { }
    floatData(std::string data) : d2_(data) { }

    explicit operator int() const {
        if(d2_ == "")
            return d1_;
        else
            return d2_ == "zero" ? 0 : d2_ == "one" ? 1 : -1;
    }

private:
    int d1_ { 0 };
    std::string d2_ { "" };
};

int main(){
    int v = 42;
    floatData f1(v);
    floatData f2("one");
    std::cout << f1 << " "; //LINE-1
    std::cout << (int)f2 << " "; //LINE-2
    std::cout << static_cast<int>(f1) << " "; //LINE-3
    std::cout << int(f2) << " "; //LINE-4
    return 0;
}
```

Which of the following lines will generate compiler error/s?

- a) LINE-1
- b) LINE-2
- c) LINE-3
- d) LINE-4

Answer: a)

Explanation:

The explicit typecast operator to `int` requires explicit conversion. At LINE-1, the compiler attempts implicit conversion when passing the object directly to `std::cout`, which is not allowed. The remaining cases use explicit typecasting, which is valid.

Intentionally kept as MSQ

Question 9

Consider the following code segment (in C++11).

[MCQ, Marks 2]

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <string>

template<typename T>
void process(std::vector<T>& tVec){
    struct compare{
        bool operator()(T a, T b){
            return a.size() < b.size();
        }
    };
    sort(tVec.begin(), tVec.end(), compare());    //LINE-1
}

int main() {
    std::vector<std::string> animals {"elephant", "cat", "tiger", "ox"};
    process(animals);
    for(auto& a : animals)
        std::cout << a << " ";
    return 0;
}
```

What will be the output?

- a) ox cat tiger elephant
- b) elephant tiger cat ox
- c) cat ox tiger elephant
- d) compiler error at LINE-1: compare is local

Answer: a)

Explanation:

The functor `compare` sorts the strings by their length in ascending order. Lengths: `ox` (2), `cat` (3), `tiger` (5), `elephant` (8). So the sorted order is `ox cat tiger elephant`. C++11 allows local functor declarations, so LINE-1 is valid.

Programming Questions

Question 1

Consider the program below (in C++11).

- Fill in the blank at LINE-1 with the appropriate template declaration.
- Fill in the blank at LINE-2 with an appropriate parameter for function `createShape`.
- Fill in the blank at LINE-3 to complete the `return` statement.

The program must satisfy the given test cases.

Marks: 3

```
#include <iostream>
#include <utility>

class Shape {
public:
    virtual void draw() = 0;
};

class Circle : public Shape {
private:
    int radius;
public:
    Circle(const int& r) : radius(r) { std::cout << "lvalue "; }
    Circle(int&& r) : radius(r) { std::cout << "rvalue "; }
    void draw() override { std::cout << "Circle:" << radius << " "; }
};

class Square : public Shape {
private:
    int side;
public:
    Square(const int& s) : side(s) { std::cout << "lvalue "; }
    Square(int&& s) : side(s) { std::cout << "rvalue "; }
    void draw() override { std::cout << "Square:" << side << " "; }
};

----- //LINE-1
T createShape(_____) { //LINE-2
    return _____; //LINE-3
}

int main() {
    int r, s;
    std::cin >> r >> s;

    auto c1 = createShape<Circle>(r);
    auto c2 = createShape<Circle>(std::move(r));
    auto sq1 = createShape<Square>(s);
    auto sq2 = createShape<Square>(std::move(s));
    std::cout << std::endl;
```

```

        c1.draw();
        c2.draw();
        sq1.draw();
        sq2.draw();
        return 0;
}

```

Public 1

Input:
5 10
Output:
lvalue rvalue lvalue rvalue
Circle:5 Circle:5 Square:10 Square:10

Public 2

Input:
7 4
Output:
lvalue rvalue lvalue rvalue
Circle:7 Circle:7 Square:4 Square:4

Private

Input:
12 6
Output:
lvalue rvalue lvalue rvalue
Circle:12 Circle:12 Square:6 Square:6

Answer:

```

LINE-1:  template<typename T, typename U>
LINE-2:  U&& param
LINE-3:  T(std::forward<U>(param))

```

Explanation:

Since both the return type and parameter type depend on templates, the function must be declared with two template parameters: `template<typename T, typename U>`

The parameter is a universal reference: `U&& param`.

Finally, the function forwards the parameter perfectly into the constructor:

```
return T(std::forward<U>(param)).
```

Question 2

Consider the following program (in C++11).

- The function `findMinLength` must return the length of the shortest string among the given arguments.
- Fill in the blanks at LINE-1 and LINE-3 with appropriate template definitions.
- Fill in the blanks at LINE-2 and LINE-4 with appropriate parameters for `findMinLength` function.

The program must satisfy the sample input and output.

Marks: 3

```
#include <iostream>
#include <string>
#include <algorithm>

----- //LINE-1
size_t findMinLength(-----){ return str.size(); } //LINE-2

----- //LINE-3
size_t findMinLength(-----){
    return std::min(str.size(), findMinLength(strings...)); //LINE-4
}

int main(){
    std::string a, b, c;
    std::cin >> a >> b >> c;
    std::cout << findMinLength(a, b, c);
    return 0;
}
```

Public 1

Input:
apple mango banana
Output:
5

Public 2

Input:
red green blue
Output:
3

Private

Input:
elephant cat hippopotamus
Output:
3

Answer:

```
LINE-1:  template <typename T>
LINE-2:  const T& str
LINE-3:  template <typename T, typename... Tail>
LINE-4:  const T& str, const Tail&... strings
```

Explanation:

For the base case (at LINE-1), the template definition is a single type parameter, so the function parameter is: `const T& str`, which returns the size of the given string.

For the recursive case (at LINE-3), the template definition must accept one string and a parameter pack of remaining strings, so it is:

```
template <typename T, typename... Tail>
```

The function parameters (at LINE-4) are:

```
const T& str, const Tail&... strings
```

The recursive step computes the minimum of the size of the first string and the result of calling `findMinLength` on the rest of the arguments. Hence the function ultimately returns the length of the shortest string among all arguments.

Question 3

Consider the following program that implements a recursive lambda function to find out the factorial of a number n , where $n \geq 0$.

- Fill in the blank at LINE-1 to declare the signature of `recFact` as `std::function`.
- Fill the blank at LINE-2 to complete the definition of lambda function `recFact`.
- Fill the blank at LINE-3 to complete the definition of lambda expression `print`, which calls `recFact`.

The program must satisfy the sample input and output.

Marks: 3

```
#include <iostream>
#include <functional>

int main(){
    _____; //LINE-1
    recFact = _____ { //LINE-2
        return n <= 1 ? 1 : n * recFact(n - 1);
    };
    auto print = _____ { //LINE-3
        std::cout << recFact(n);
    };
    int n;
    std::cin >> n;
    print(n);
    return 0;
}
```

Public 1

Input: 5
Output: 120

Public 2

Input: 0
Output: 1

Private

Input: 7
Output: 5040

Answer:

LINE-1: `std::function<int(int)> recFact`
LINE-2: `[&recFact](int n) -> int`
LINE-3: `[&recFact](int n)`

Explanation:

At LINE-1, we declare the recursive lambda's type using `std::function`: `std::function<int(int)> recFact`.

At LINE-2, the recursive lambda captures itself by reference and takes one integer parameter:

```
recFact = [&recFact](int n) -> int {  
    return n <= 1 ? 1 : n * recFact(n - 1);  
};
```

At LINE-3, the `print` lambda captures `recFact` and invokes it with the given argument:

```
auto print = [&recFact](int n) {  
    std::cout << recFact(n);  
};
```

Thus, the program correctly computes the factorial of the input number.